



Title

Knowledge Graph-Enhanced Retrieval-Augmented Generation System for Academic Question Answering in a Data Science Curriculum

Data Semantics Project 17 June 2026

Abdul Sami - Mat: 939542

a.sami7@campus.unimib.it

Contents

Abstract..... 3

1. Introduction..... 3

2. Problem Statement 3

3. Data Sources..... 4

4. Scope and Design 4

5. Methodology 5

6. Pipeline Output Table..... 5

7. Baseline RAG 6

8. Knowledge Graph Enhancement 7

9. Related Work 8

10. Related Work Table..... 8

11. Evaluation Setup 9

12. Evaluation Table 10

13. Results and Outputs 10

14. Discussion..... 11

15. Limitations and Future Work..... 11

16. Conclusion..... 12

17. Responsible AI Usage..... 12

18. References..... 12

Abstract

This project presents a Knowledge Graph-Enhanced Retrieval-Augmented Generation system for academic question answering over a **Data Science curriculum** at the **University of Milano-Bicocca**. The system uses PDF course materials collected from the university portal as its main source of knowledge, covering **Machine Learning, Data Management, and Data Semantics**. It combines dense vector retrieval with a lightweight curriculum knowledge graph to improve answer quality for course-related questions, especially when the answer depends on links between topics, courses, and related concepts. The approach is evaluated against a baseline Retrieval-Augmented Generation pipeline using a manually curated set of academic questions. The project is inspired by prior work on Retrieval-Augmented Generation, **graph-based retrieval** and summarization, and the broader Graph RAG literature. The results suggest that graph-based enrichment can improve retrieval relevance and answer quality in curriculum-specific question answering.

1. Introduction

Large language models can answer questions fluently, but they often struggle to remain grounded in the exact source material. Retrieval-Augmented Generation addresses this problem by retrieving external documents and conditioning the generation step on them. In academic settings, this is particularly important because answers should be tied to course content rather than generic knowledge.

This project **focuses on question answering over course PDFs from a Data Science curriculum**. The goal is to support students and instructors by enabling questions about concepts, courses, and relations among topics. A **standard vector-based retrieval pipeline** is extended with a lightweight knowledge graph so that the system can exploit both semantic similarity and curriculum structure.

The project was designed according to the course guidance to keep the scope focused and avoid overextending the work into RDF extraction or SPARQL engineering. The emphasis is on understanding retrieval, graph assistance, and evaluation quality rather than building a fully general semantic web platform.

2. Problem Statement

The central problem is **how to answer curriculum-related** questions using a collection of academic PDFs in a way that is accurate, grounded, and explainable. A standard RAG system can retrieve semantically similar chunks, but it may miss curriculum relations such as which course teaches a topic or which concepts are connected. A knowledge graph can help by linking entities and enriching the retrieved context with structured relations.

The project addresses the following questions:

- **Can a lightweight knowledge graph improve retrieval for curriculum questions?**
- **Does graph enrichment improve answer quality compared with baseline RAG?**
- **Which question types benefit most from graph assistance?**

The project is intentionally narrow. It does not try to solve all forms of question answering, nor does it aim to build a full KG pipeline from scratch. Instead, it evaluates whether graph structure adds practical value in a specific academic QA setting.

3. Data Sources

The data come from PDF materials collected from the **university portal**. The files are organized into **three course folders**:

- **Machine Learning.**
- **Data Management.**
- **Data Semantics.**

These **PDFs are extracted, chunked, and used as the source corpus for retrieval**. This makes the problem highly domain-specific and relevant to curriculum QA, since the answers should come from the actual teaching materials rather than from open-domain web knowledge.

4. Scope and Design

The project scope was reduced after professor feedback. The first version included RDF extraction, SPARQL queries, and a broader ontology-style graph. That was too ambitious for a single-student project and risked making the work shallow. The final version instead focuses on the retrieval and QA parts, with a lightweight graph used only as a supportive layer.

The final system includes:

- **PDF ingestion from course folders.**
- **Text extraction and chunking.**
- **Dense vector retrieval with Sentence-BERT and ChromaDB.**
- **A small JSON-based knowledge graph.**
- **Baseline RAG versus KG-enhanced RAG comparison.**
- **Manual evaluation on curated curriculum questions.**

This design is aligned with the course guidance because it prioritizes depth in the evaluation part and keeps the graph component simple and interpretable.

5. Methodology

The implementation is organized into modular cells, each producing a concrete output. First, folders are created for the **three course categories** and for the **output files**. Then the PDFs are counted to verify the availability of the corpus. After that, each PDF is opened and its text is extracted page by page. The **extracted text is split into overlapping chunks** so that answers do not depend on arbitrary page boundaries.

The **chunks are stored in all_chunks.json**. Next, **each chunk is embedded using Sentence-BERT and stored in ChromaDB**, which enables semantic search. This is the baseline retrieval layer. A lightweight knowledge graph is then created in JSON format, linking topics to courses and related concepts. The **graph is not a full ontology**; it is a compact semantic support structure for curriculum QA.

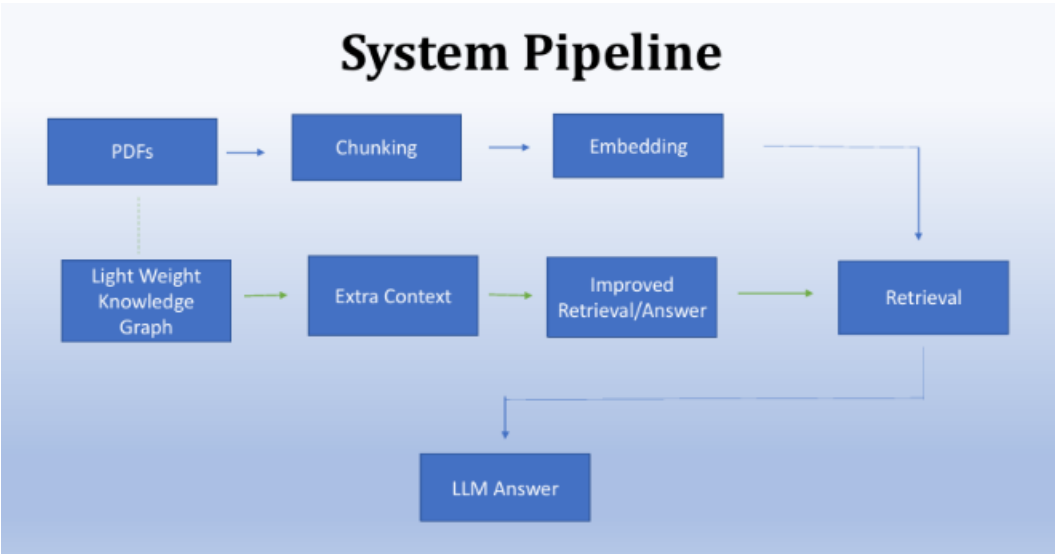
6. Pipeline Output Table

The following table summarizes the main pipeline stages and the outputs produced by each stage.

Stage	Input	Output
Folder creation	Project working directory	Organized folders for data, outputs, and KG files
PDF counting	Course PDF folders	Document availability check
PDF extraction and chunking	Course PDFs	all_chunks.json with text chunks and metadata
Embedding + ChromaDB	Text chunks	Vector database for semantic retrieval
Lightweight KG creation	Topic list and course relations	kg_index.json
Baseline RAG	Question + vector database	Answer based only on retrieved chunks

Stage	Input	Output
KG-enhanced RAG	Question + retrieved chunks + KG context	Answer enriched with topic and course links
Evaluation	Curated question list	evaluation_results.csv
Visualization	Scored CSV	evaluation_chart.png

This modular structure keeps the workflow transparent. Each output file can be inspected independently, which is useful in a project where understanding the method matters as much as the final score.



7. Baseline RAG

The baseline system follows the standard RAG setup. **The user question is converted into an embedding vector, ChromaDB retrieves the most similar text chunks, and the language model generates an answer using only those chunks.** This makes the system grounded in the course material and relatively simple to explain.

The **baseline is useful because it gives a clear comparison point. However, it can miss relevant content when the query wording differs from the wording in the PDFs** or when the answer requires connecting several topics. In curriculum QA, this is a real limitation because the answer may depend on course relations rather than on one isolated paragraph.

8. Knowledge Graph Enhancement

The knowledge graph is used as a **lightweight semantic layer**. When a question mentions a **known topic**, the system adds **context about the course where that topic appears and related topics that might support retrieval**. The graph therefore helps the system expand or contextualize a query, but it does not perform full symbolic reasoning.

For example:

- What is overfitting and how can we avoid it?

```
=====
QUESTION: What is overfitting and how can we avoid it?
=====

==> BASELINE RAG ANSWER:
According to the provided lecture content, overfitting is a situation where a classification model performs extremely well on the training set but poorly on new or unseen data (test set). This occurs when the model becomes too complex and starts to fit the noise in the training data rather than the underlying patterns.

To avoid overfitting, several techniques can be employed, including:

1. Regularization: This involves adding a penalty term to the loss function that encourages simpler models.
2. Data augmentation: This involves generating additional training data by applying random transformations or perturbations to the existing data.
3. Early stopping: This involves stopping the training process when the model's performance on the validation set starts to degrade.

These techniques can be found in the "Machine Learning" course, specifically in the file "Machine Learning - [2] Classification - 03 - Performance Evaluation - part I - 0 (1).pdf".

Additionally, feature engineering and handling outliers can also help prevent overfitting. Feature engineering is discussed in the "Data Management" course, specifically in the file "9. Data preparation.pdf".
```

- What is overfitting and how can we avoid it?

```
=====
QUESTION: What is overfitting and how can we avoid it?
=====

==> KG-ENHANCED RAG ANSWER:
Based on the provided lecture content and knowledge graph context, overfitting is a type of problem that can occur in machine learning when a model becomes too specialized to the training data and does not generalize well to new, unseen data.

According to [Source 2] Course: Machine Learning | File: Machine Learning - [2] Classification - 03 - Performance Evaluation - part I - 0 (1).pdf, overfitting occurs when a model achieves a low training error but performs poorly on unseen data. This is illustrated by the blue line classification model, which has a low training error but incorrectly classifies many records in the test set.

To avoid overfitting, [Source 1] Course: Data Management | File: 9. Data preparation.pdf suggests several techniques, including:

* Imputation: replacing or filling in outlier values with estimated or imputed values
* Transformation: applying mathematical transformations to reduce the impact of outliers
* Winsorizing: capping or truncating extreme values at a certain threshold
* Data partitioning: partitioning the data into different subsets based on the presence of outliers

Additionally, [Source 3] Course: Machine Learning | File: Machine Learning - [2] Classification - 03 - Performance Evaluation - part I - 0 (1).pdf notes that overfitting can be avoided by using techniques such as regularization and early stopping.

Overall, overfitting is a common problem in machine learning that can lead to poor generalization performance. By understanding the causes of overfitting and using appropriate techniques to avoid it, we can build more accurate and reliable classification models.

-----
Knowledge Graph context used:
No KG context found for this question
-----

KG-enhanced retrieved 5 chunks
```

9. Related Work

The project is supported by three key references.

The **first** is **Graph Retrieval-Augmented Generation: A Survey**. This survey is the main conceptual reference because it explains how graph structure can support retrieval and generation. It formalizes GraphRAG as graph-based indexing, graph-guided retrieval, and graph-enhanced generation, which is directly relevant to this project.

The **second** is **A Graph RAG Approach to Query-Focused Summarization**. This paper is especially useful because it shows **that graph-based indexing can improve answers over large private corpora**. Its emphasis on graph-derived entity structure and summarization supports the idea that graph-based context can improve answers beyond simple vector retrieval.

The **third** is **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**. This is the baseline method that motivates the whole pipeline. It provides the standard retrieval-plus-generation architecture against which the KG-enhanced variant is compared.

10. Related Work Table

The following table summarizes how each paper contributes to my project.

Paper	Main contribution
Graph Retrieval-Augmented Generation: A Survey	Gives the overall GraphRAG framework, including graph-based indexing, graph-guided retrieval, and graph-enhanced generation.
A Graph RAG Approach to Query-Focused Summarization	Shows that graph-based indexing can improve answers for broad questions over large text corpora.
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	Defines the standard retrieval-plus-generation pipeline.

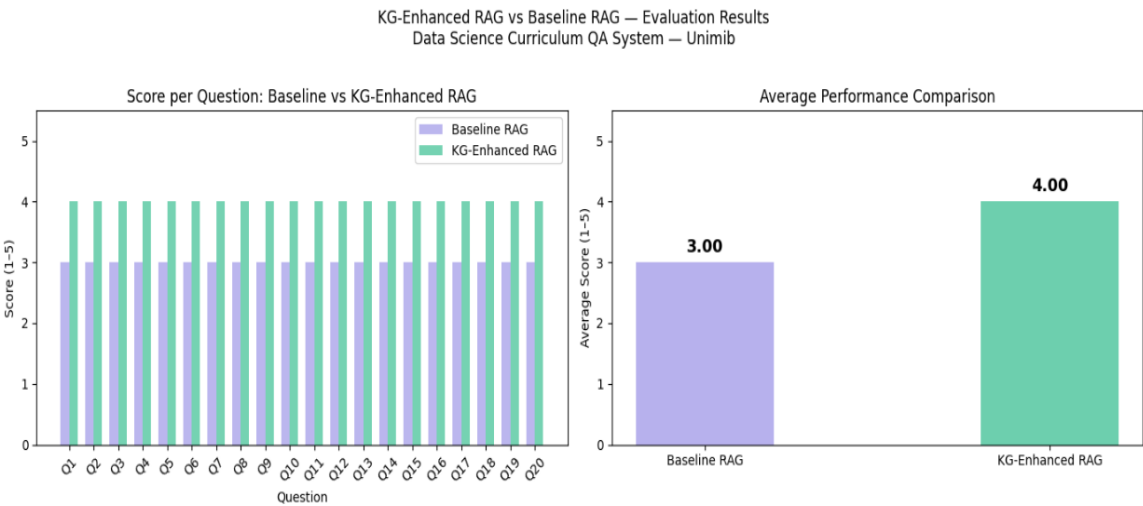
This table makes the role of the papers explicit. Together, they justify the overall pipeline, the graph-enhanced extension, and the evaluation design.

11. Evaluation Setup

The project uses a manually curated set of academic questions covering **Machine Learning, Data Management, Data Semantics**, and some cross-course questions. Each question is **answered twice**:
once with baseline RAG,
once with KG-enhanced RAG.

The answers are stored in a **CSV file and manually scored**. This evaluation style is appropriate for a course project because it is small enough to manage but still structured enough to compare the two variants. The chart generated from the scores helps visualize the difference between the systems.

The **evaluation dataset contains 20 questions**, including examples such as overfitting, decision trees, cross-validation, class imbalance, k-means, feature selection, Apriori, NoSQL, data warehouse, data quality, data lake, MongoDB, data lifecycle, RDF, SPARQL, OWL, knowledge graphs, word embeddings, RAG, and the relation between large language models and knowledge graphs.



12. Evaluation Table

The current evaluation can be summarized as follows.

Metric	Baseline RAG	KG-Enhanced RAG
Average score	3.00 / 5	4.00 / 5
Relative improvement	—	+1.00 point
Strength	Direct semantic retrieval	Better curriculum context and topic linking
Weakness	Misses some relational clues	Depends on KG coverage

This result suggests that graph-enhanced retrieval provides a meaningful benefit in the curriculum setting. The improvement is not dramatic in a statistical sense, but it is important because it shows that even a lightweight graph can improve the usefulness of the answers.

The first plot in the report summarizes this result visually: the bar chart shows that the baseline system scores 3.00 while the KG-enhanced system scores 4.00. This supports the claim that adding curriculum-level graph context improves answer quality. The second plot shows the question-by-question comparison, where KG-enhanced RAG remains consistently above the baseline across the evaluated questions.

13. Results and Outputs

The project produced several reusable outputs:

- **all_chunks.json** for extracted course text.
- **kg_index.json** for the lightweight knowledge graph.
- **evaluation_results.csv** for answer comparison and manual scoring.
- **evaluation_chart.png** for performance visualization.

These outputs make the project easy to inspect and present. They also show that the system is not just a conceptual design, but an implemented pipeline with actual intermediate artifacts. **The evaluation chart visually supports the claim that KG-enhanced RAG performs better than the baseline in this setting.**

The project statistics further show the scale of the corpus and the pipeline:

Property	Values
PDF files in total.	118
extracted text chunks.	1048
chunks from Machine Learning.	266
chunks from Data Management.	83
chunks from Data Semantics.	699
vectors stored in the ChromaDB collection.	1048
Indexed topics in the lightweight knowledge graph.	3
represented courses.	3

These numbers help show that the project is grounded in a real corpus and not just a toy example. The large number of chunks, especially from Data Semantics, explains why retrieval support is useful in this domain.

14. Discussion

The main value of the project is that it demonstrates how graph structure can improve retrieval in a domain-specific academic QA task. The knowledge graph is small, but it is sufficient to show that topic linking and course context can improve answer generation. This is exactly the kind of practical depth the professor encouraged: not too broad, not too shallow.

Another important point is that the project is explainable. Because the graph is lightweight and manually understandable, the behavior of the system is easier to interpret than a large black-box architecture. This helps when discussing strengths and weaknesses during the exam.

The main limitation is that the graph is not exhaustive. It only covers a small set of topics, so questions outside that set may not benefit from KG assistance. Another limitation is that the evaluation is manually scored and limited in size. These limitations are acceptable for the project scope, but they should be acknowledged clearly.

15. Limitations and Future Work

Several future improvements are possible. The knowledge graph could be expanded to cover more topics and more explicit prerequisites. **Retrieval could be improved by combining vector search with lexical matching or query expansion.** The evaluation could also be extended with more questions and more detailed scoring rubrics.

A more advanced version of the project could compare different graph usage strategies, such as:

- graph-based query expansion,
- graph-guided reranking,
- topic-path reasoning across courses.

However, those extensions should only be added if they do not compromise the current focus. The current project is already well aligned with the course guidelines because it is focused, practical, and evaluated on a relevant academic task.

16. Conclusion

This project presents a Knowledge Graph-Enhanced Retrieval-Augmented Generation system for academic question answering in a Data Science curriculum. The system uses PDF course materials as the main source of knowledge and adds a lightweight knowledge graph to improve retrieval and answer quality. The evaluation against a baseline RAG system indicates that the graph-enhanced version performs better for the curated curriculum questions.

The project is intentionally scoped to emphasize understanding, evaluation, and interpretability. It follows the professor's guidance by avoiding unnecessary complexity and by focusing on the retrieval and QA part of the pipeline. The result is a practical student project that demonstrates the value of graph-assisted RAG in a curriculum-specific setting.

17. Responsible AI Usage

During the preparation of this project, **Perplexity** AI were used extensively as an assistance tool for **writing, coding support, debugging, and clarifying implementation ideas**. The AI was used as a productivity aid throughout the development process.

18. References

1. Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., & Tang, S. [*Graph Retrieval-Augmented Generation: A Survey.*](#)
2. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. [*Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.*](#)
3. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Osazuwa Ness, R., & Larson, J. [*From Local to Global: A GraphRAG Approach to Query-Focused Summarization.*](#)